PROYECTO 2

BiblioComunidad

Sistema de gestión para una biblioteca comunitaria

Dato	Información
Asignatura	Programación (G02)
Programa	Ingeniería de Datos e Inteligencia Artificial
Integrantes	Juan Felipe Valencia Melenje / Juan José Domínguez Botina
Docente	Breyner Posso M.Sc.
Entrega	Informe final e implementación - 15 de octubre de 2026

Resumen. BiblioComunidad es un MVP local desarrollado en Python para administrar lectores, ejemplares físicos y préstamos. La solución aplica POO, encapsulamiento, asociaciones entre objetos, CRUD, reglas de negocio, persistencia en JSON, una interfaz de consola y una interfaz gráfica con Tkinter que reutilizan la misma lógica de gestión.

1. Análisis del problema

Una biblioteca comunitaria necesita diferenciar cada copia física mediante un código único, conocer su disponibilidad, controlar qué lector tiene cada ejemplar y conservar el historial de préstamos y devoluciones. El sistema debe rechazar operaciones inválidas sin alterar el estado anterior.

1.1 Propósito y alcance

- Gestionar lectores mediante registro, listado, búsqueda, actualización y eliminación cuando las reglas lo permitan.
- Gestionar ejemplares físicos con código único, título, autor y disponibilidad.
- Consultar ejemplares por código y por título parcial.
- Registrar préstamos y devoluciones, conservando el historial.
- Impedir que un ejemplar tenga más de un préstamo ACTIVO.
- Persistir datos y asociaciones en JSON.
- Permitir las operaciones desde consola y desde Tkinter usando las mismas funciones de gestión.
- Excluir multas, vencimientos, pagos, autenticación, bases de datos, servicios web y reservas previas.

1.2 Entradas, salidas y reglas

Elemento	Definición
Entradas	Lector: documento, nombre, teléfono. Ejemplar: código, título, autor. Préstamo: ID, lector, ejemplar y días. Devolución: ID del préstamo. Búsquedas y opciones de menú.
Salidas	Listados, fichas, disponibilidad, resultados de búsqueda, confirmaciones y mensajes de rechazo.
Colecciones	Listas de objetos Lector, Ejemplar y Prestamo, cargadas y guardadas en JSON.
Operación central	Registrar préstamo, consultar historial y registrar devolución.

Regla	Criterio

R1	Documento de lector, código de ejemplar e ID de préstamo deben ser no vacíos y únicos.
R2	Un ejemplar se crea disponible y solo puede tener un préstamo ACTIVO a la vez.
R3	Un préstamo requiere lector y ejemplar existentes, ejemplar disponible y días como entero mayor que cero.
R4	Al devolver un préstamo ACTIVO, pasa a DEVUELTO y el ejemplar vuelve a disponible.
R5	No se permite devolver dos veces el mismo préstamo.
R6	No se elimina físicamente un lector o ejemplar si tiene historial de préstamos asociado.
R7	Los identificadores no se editan; se actualizan solo datos descriptivos.
R8	Una operación rechazada conserva el estado anterior.
R9	Si el JSON no existe, se inicia con colecciones vacías.
R10	Si el JSON no puede leerse, se informa el error y no se sobrescribe automáticamente.

2. Diseño orientado a objetos

Clase / módulo	Responsabilidad	Atributos principales	Métodos / funciones principales
Lector	Representar una persona registrada.	_documento, _nombre, _telefono	obtener_*(), actualizar_datos(), obtener_informacion()
Ejemplar	Representar una copia física individual.	_codigo, _titulo, _autor, _disponible	obtener_*(), esta_disponible(), actualizar_datos(), marcar_prestado(), marcar_disponible()
Prestamo	Relacionar un lector con un ejemplar durante una cantidad de días.	_id_prestamo, _lector, _ejemplar, _dias, _estado	obtener_*(), esta_activo(), marcar_devuelto(), obtener_informacion()
gestion_biblioteca.py	Administrar colecciones y reglas que involucran varios objetos.	-	CRUD, búsquedas, registrar_prestamo(), devolver_prestamo()
persistencia.py	Guardar y reconstruir datos y asociaciones.	archivo JSON	guardar_datos(), cargar_datos()
app_consola.py / app_gui.py	Capturar entradas y mostrar resultados.	-	Reutilizan funciones de gestión; no duplican reglas.

Corrección aplicada del avance: los atributos se marcan con un solo guion bajo, por ejemplo _documento y _codigo, en lugar de doble guion bajo.

2.1 UML y asociaciones

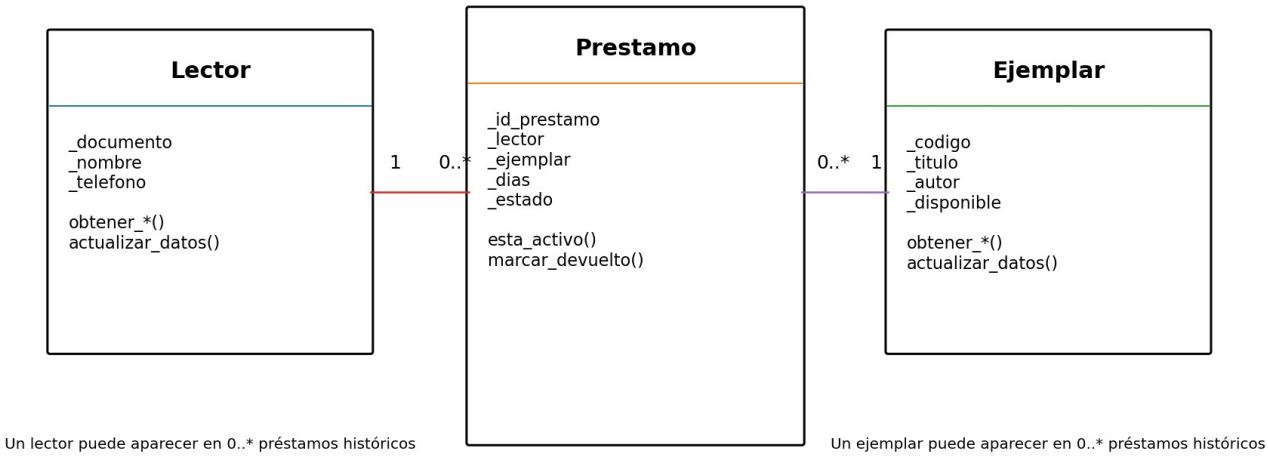


Figura 1. Prestamo se asocia con exactamente un Lector y un Ejemplar. Un lector o un ejemplar puede aparecer en cero o muchos préstamos históricos.

Relación	Multiplicidad	Interpretación
Lector - Prestamo	1 a 0..*	Cada préstamo pertenece a un lector; un lector puede tener varios préstamos históricos.
Ejemplar - Prestamo	1 a 0..*	Cada préstamo usa un ejemplar; un ejemplar puede aparecer en varios préstamos históricos, pero solo uno puede estar ACTIVO.

3. Diseño de operaciones

Operación	Responsable	Entradas	Validaciones	Resultado / estado
Registrar lector	registrar_lector	documento, nombre, teléfono	Campos no vacíos; documento único	Agrega Lector.
Listar lectores	listar_lectores	-	-	Devuelve lista.
Buscar lector	buscar_lector	documento	No vacío	Lector o None.
Actualizar lector	actualizar_lector	documento + datos	Existe; datos válidos	Modifica nombre/teléfono.
Eliminar lector	eliminar_lector	documento	Existe; sin historial	Elimina o rechaza.
Registrar ejemplar	registrar_ejemplar	código, título, autor	Código único; datos válidos	Agrega disponible=True.
Listar ejemplares	listar_ejemplares	-	-	Devuelve lista.
Buscar por código	buscar_ejemplar	código	No vacío	Ejemplar o None.
Buscar por título	buscar_por_titulo	texto	Texto no vacío	Lista de coincidencias.
Actualizar ejemplar	actualizar_ejemplar	código + datos	Existe; datos válidos	Actualiza título/autor.
Eliminar ejemplar	eliminar_ejemplar	código	Existe; sin historial	Elimina o rechaza.
Registrar préstamo	registrar_prestamo	ID, lector, ejemplar, días	ID único; relaciones existen; días > 0; disponible	Crea ACTIVO y marca ejemplar prestado.
Listar préstamos	listar_prestamos	-	-	Devuelve historial.
Buscar préstamo	buscar_prestamo	ID	No vacío	Prestamo o None.
Listar activos	listar_activos	-	-	Devuelve solo ACTIVO.
Devolver	devolver_prestamo	ID	Existe y está ACTIVO	Pasa a DEVUELTO y libera ejemplar.

4. Contratos principales

Método / función	Retorno	Contrato
buscar_lector(documento)	Lector | None	No modifica estado; None si no existe.
buscar_ejemplar(codigo)	Ejemplar | None	No modifica estado; None si no existe.
buscar_por_titulo(texto)	list[Ejemplar]	Búsqueda parcial sin distinguir mayúsculas; puede devolver lista vacía.
registrar_prestamo(...)	(bool, mensaje, Prestamo | None)	Solo modifica estado después de validar todas las reglas.
devolver_prestamo(id)	(bool, mensaje)	Solo cambia ACTIVO a DEVUELTO; una segunda devolución se rechaza.
eliminar_lector / eliminar_ejemplar	(bool, mensaje)	Solo elimina si no existe ningún préstamo histórico asociado.
guardar_datos(...)	bool / excepción controlada	Serializa colecciones y relaciones mediante identificadores.
cargar_datos()	3 colecciones	Reconstruye lectores y ejemplares antes de reconstruir préstamos y sus referencias.

5. Colaboración entre objetos

5.1 Registrar un préstamo

1. La interfaz recibe ID del préstamo, documento del lector, código del ejemplar y cantidad de días.
2. gestion_biblioteca.py valida que el ID sea único y localiza el Lector y el Ejemplar.
3. Se validan existencia, disponibilidad y días mayores que cero. Si algo falla, no se modifica ningún objeto.
4. Se crea Prestamo en estado ACTIVO y el Ejemplar se marca como no disponible.
5. La operación se agrega al historial y la interfaz solicita el guardado del estado actualizado.

5.2 Registrar una devolución

6. La interfaz recibe el ID y busca el Prestamo.
7. Si no existe o ya está DEVUELTO, la operación se rechaza sin cambios.
8. Prestamo cambia de ACTIVO a DEVUELTO.
9. El Ejemplar relacionado vuelve a disponible y se guarda el nuevo estado.

6. Persistencia en JSON

El archivo datos_biblioteca.json guarda datos simples. Los préstamos no guardan objetos completos: almacenan el documento del lector y el código del ejemplar. Al cargar, primero se crean Lector y Ejemplar; después se reconstruyen Prestamo enlazando esos identificadores con los objetos ya existentes.


```
{
  "lectores": [{"documento": "1001", "nombre": "Ana Torres", "telefono": "3001112233"}],
  "ejemplares": [{"codigo": "E001", "titulo": "Cien años de soledad", "autor": "Gabriel García Márquez", "disponible": false}],
  "prestamos": [{"id": "P001", "lector": "1001", "ejemplar": "E001", "dias": 7, "estado": "ACTIVO"}]
}
```


6.1 Manejo de errores

- Si el archivo no existe, cargar_datos() retorna tres colecciones vacías.
- Si el archivo no puede leerse o contiene referencias inconsistentes, se lanza ErrorPersistencia y la interfaz informa el problema.
- Cuando la carga inicial falla, las interfaces no sobrescriben automáticamente el archivo dañado.
- El guardado normal se realiza de forma atómica mediante un archivo temporal y os.replace().
- La disponibilidad se recalcula a partir de los préstamos ACTIVO al cargar para mantener coherencia.

7. Consola e interfaz gráfica

La consola y la GUI utilizan las mismas funciones de gestion_biblioteca.py. La diferencia está únicamente en la captura de entradas y la presentación de resultados. La GUI usa Tkinter y un Notebook con tres módulos: Lectores, Ejemplares y Préstamos.

Módulo	Operaciones disponibles en consola y GUI
Lectores	Registrar, listar, buscar, actualizar y eliminar.
Ejemplares	Registrar, listar, buscar por código, buscar por título, actualizar y eliminar.
Préstamos	Registrar, listar, buscar, listar activos y devolver.

8. Registro de cambios frente a la entrega inicial

Cambio	Aplicación
Atributos encapsulados	Se sustituyó el doble guion bajo por un solo guion bajo en Lector, Ejemplar y Prestamo, siguiendo la retroalimentación recibida.
Implementación	Se completaron las clases del dominio, funciones de gestión, consola y GUI.
Persistencia	Se implementó guardado/carga JSON, recuperación de asociaciones y manejo de archivo inexistente o ilegible.
Pruebas	Se creó pruebas_integracion.py y se ejecutaron verificaciones de CRUD, conflictos, devoluciones, eliminación y persistencia.
Datos de demostración	Se agregaron 5 lectores, 6 ejemplares y 5 préstamos con estados ACTIVO/DEVUELTO.

9. Pruebas ejecutadas

Las pruebas se ejecutaron sobre la lógica real del sistema. Además, todos los archivos .py compilaron correctamente y la interfaz Tkinter fue iniciada en un entorno gráfico de prueba con 5 lectores, 6 ejemplares y 5 préstamos cargados.

ID	Condición inicial	Entrada	Esperado	Obtenido	Estado
P01	Colecciones vacías	Registrar lector 1001	Se crea	Se creó y quedó disponible para búsqueda	APROBADA
P02	Existe lector 1001	Registrar 1001 otra vez	Se rechaza sin cambios	Se rechazó y quedó 1 lector	APROBADA
P03	Existe lector 1001	Buscar 9999	None / no encontrado	Retornó None	APROBADA
P04	Lista vacía	Listar	Lista vacía sin error	Se obtuvo lista vacía sin error	APROBADA
P05	E001 disponible	Préstamo con días 0	Rechazar; E001 sigue disponible	Se rechazó y no cambió disponibilidad	APROBADA
P06	Lector 1001 y E001 disponibles	P001 por 7 días	Crear ACTIVO; E001 no disponible	Se creó ACTIVO y E001 quedó prestado	APROBADA
P07	E001 con P001 ACTIVO	Segundo préstamo de E001	Rechazar conflicto	Se rechazó; historial no cambió	APROBADA
P08	P001 ACTIVO	Devolver P001	DEVUELTO; E001 disponible	Cambió a DEVUELTO y E001 se liberó	APROBADA
P09	P001 DEVUELTO	Devolver P001 otra vez	Rechazar	Se rechazó sin cambios	APROBADA
P10	E002 sin historial	Eliminar E002	Eliminar	E002 fue eliminado	APROBADA
P11	E001 con historial	Eliminar E001	Rechazar	Se rechazó y E001 permanece	APROBADA
P12	Datos guardados	Guardar, cerrar y cargar	Recuperar datos, estados y asociaciones	Se recuperaron y las referencias apuntaron a los objetos correctos	APROBADA
P13	Lector 1001 con historial	Eliminar lector 1001	Rechazar	Se rechazó y el lector permanece	APROBADA

Resultado general: 16 verificaciones internas ejecutadas y aprobadas; 13 casos funcionales principales documentados en esta matriz.

10. Implementación final y ejecución

Archivo	Responsabilidad
lector.py	Modelo Lector.
ejemplar.py	Modelo Ejemplar.
prestamo.py	Modelo Prestamo y estados ACTIVO/DEVUELTO.
gestion_biblioteca.py	CRUD, búsquedas y reglas de negocio.
persistencia.py	JSON, carga, guardado y recuperación de asociaciones.
app_consola.py	Menús de consola.
app_gui.py	Interfaz gráfica Tkinter.
datos_biblioteca.json	Datos de demostración.
pruebas_integracion.py	Pruebas automáticas.
README.md	Instrucciones de ejecución.

Consola: python app_consola.py

GUI: python app_gui.py

Pruebas: python pruebas_integracion.py

Dependencias adicionales: ninguna. Se utilizan módulos de la biblioteca estándar de Python, incluyendo json, os, tempfile y tkinter.

11. Fuentes y herramientas de apoyo

- Enunciado Programming-G2_2026-2_Proyecto-2_v2.pdf y materiales de clase de Programación (G02) sobre POO, asociaciones, persistencia JSON y Tkinter.
- Documentación oficial de Python para json, tkinter, os y tempfile.
- OpenAI ChatGPT como herramienta de apoyo para revisión de estructura, depuración y verificación.